

RAG検索チャットUI・モーダル統合・Embedding改善・

Spec • Created May 28, 2026

本仕様書は、RAGベクトル検索チャットUIの新規実装、検索機能のモーダル（Sheet）統合によるページ遷移なしの検索体験、
+ Agentによるマルチターン検索、Vercel Data Protocolによるストリーミング、Sheetコンポーネントによるインライン検索体験を軸とし、既存の互換性を保ちつつ、

RAG検索チャットUI・モーダル統合・Embedding改善・ダッシュボード統合・

1. 概要

本仕様書は、スクラップ管理アプリケーションにおける以下5つの機能改善・追加についてまとめたものである。

- RAGベクトル検索チャットUIの実装（`/search` ページ）
- RAG検索のモーダル統合（ページ遷移なしのSheet UI）
- Embeddingテキスト戦略の改善
- ダッシュボードのスクラップ一覧統合とアーカイブフィルタ化
- Codex向けプラン保存フロー移植

2. RAGベクトル検索チャットUIの実装（`/search`）

2.1 背景

スクラップが蓄積されてきたことで「あの計画どこに書いたっけ？」という曖昧な記憶から探したいニーズに対応するため、
+ embedding は既に動いているため、それを使った自然言語検索チャットUIを `/search` ページとして追加する。マルチターンの会話に対応し、同一ページ滞在中は文脈を保持するが、ページを離れたらリセットする。

2.2 設計方針

- 会話永続化: `RemembersConversations` トレイトで `agent_conversations` / `agent_conversation_messages` テーブルに蓄積
- 会話の開始: `GET /search` 時にサーバー側で `conversation_id` を事前発行（Inertia props で渡す）→ クエリは常に `continue()` で送れるようにする

- RAG: コントローラーで `whereVectorSimilarTo` によるベクトル検索を行い、取得したスクラップをプロンプトに注入してから Agent に渡す
- ストリーミング: `->stream()->usingVercelDataProtocol()` で SSE 送信。フロントは `fetch + ReadableStream` で受け取りパース（新依存なし）

2.3 バックエンド実装

`app/Ai/Agents/SearchScrapAgent.php`（新規）

`Agent`, `Conversational` を実装し `RemembersConversations` トレイトを使用。

`#[UseCheapestModel]` を付与。

```
#[UseCheapestModel]
class SearchScrapAgent implements Agent, Conversational
{
    use Promptable, RemembersConversations;

    public function instructions(): string
    {
        return <<<'TEXT'
あなたは個人のスクラップコレクションのナレッジアシスタントです。
提供された関連スクラップの内容をもとに、ユーザーの質問に日本語で簡潔に回答してください。
関連スクラップが見つからなかった場合はその旨を正直に伝えてください。
TEXT;
    }
}
```

`app/Http/Controllers/SearchController.php`（新規）

- `index()`: `agent_conversations` に新規レコードを直接 DB 挿入し、`conversationId` を `Inertia props` として返す
- `query()`: クエリを受け取り、`whereVectorSimilarTo` で最大5件取得 → プロンプトに注入 → `->continue($conversationId, as: $user)->stream()->usingVercelDataProtocol()` で返却

```
private function buildContextualPrompt(string $query, Collection $scraps): string
{
    if ($scraps->isEmpty()) {
        return $query;
    }
}
```

```

$context = $scraps
->map(fn ($s) => "### {$s->title}¥n" . ($s->summary ?? $s->content_markdown))
->implode("¥n¥n");

return "関連スクラップ:¥n{$context}¥n¥n---¥n質問: {$query}";
}

```

app/Http/Requests/SearchQueryRequest.php (新規)

query (必須・文字列) と conversation_id (必須・UUID 形式) を検証。

routes/web.php (変更)

認証ミドルウェアのグループ内に追加:

```

Route::get('/search', [SearchController::class, 'index'])->name('search');
Route::post('/search/query', [SearchController::class, 'query'])->name('search.query');

```

2.4 フロントエンド実装

resources/js/pages/search.tsx (新規)

Inertia ページコンポーネント。受け取る props: { conversationId: string }

- state: messages: Message[], input: string, isStreaming: boolean
- 送信時: fetch(SearchController.query().url, { method: 'POST', ... }) + ReadableStream
でストリーミング受信
- Vercel Data Protocol パース: 0:"text" 行を抽出してメッセージに追加
- CSRF: document.querySelector('meta[name="csrf-token"]') から取得 (Inertia 標準)
- UI: シンプルなメッセージリスト + 入力欄。送信中はローディング表示

resources/js/components/app-header.tsx (変更)

ナビゲーション配列に Search エントリを追加 (Search アイコン、search() Wayfinder ルート)。

2.5 既存コードの再利用

- whereVectorSimilarTo('embedding', \$queryText) —
フレームワーク提供。文字列を渡すと自動で embedding 生成・ベクトル検索

- `response()->eventStream()` パターン (`BackupController::bulkStream`) — SSE の参考実装
 - `SuggestScrapMetadataAgent` / `SummarizeScrapAgent` — Agent クラスの構造の参考
 - `Embeddings::fake()` / `Agent::fake()` — テストのフェイクパターン
-

3. RAG検索をモデルに統合 (ページ遷移なし)

3.1 背景

ヘッダーの検索アイコンをクリックすると `/search`

にフルページ遷移し、読書・編集の流れが途切れる。ページを離れずにRAGサーチを呼び出せるよう、Sheet `Cmd+K` でも開けるようにする。

3.2 設計方針

UIスタイル: Sheet (右スライド)

- `resources/js/components/ui/sheet.tsx` がすでに存在 (追加依存なし)
- フルハイトのチャットUIと相性が良い (Dialog は幅・高さが制約的すぎる)
- 既存の `search.tsx` は `h-[calc(100dvh-4rem)]` のフルハイト設計 → Sheet に自然にマップ

conversationId の扱い: 新エンドポイント `POST /search/init`

- 現状: `conversation_id` は `SearchQueryRequest` で `required|uuid` — クライアントだけでは完結しない
- `POST /search/init` を追加し、DBレコード作成 + UUID をJSONで返す
- ダイアログを開いたときに呼び出してIDを取得 (開くまで会話は作られない)
- 既存の `GET /search` ページ・`POST /search/query` は一切変更しない

キーボードショートカット: `Cmd+K` / `Ctrl+K`

- サイドバーが `Ctrl+B` を使っているパターン (`sidebar.tsx`) を踏襲
- `SearchDialog` 内部の `useEffect` でグローバルに `keydown` を listen

3.3 バックエンド実装

`app/Http/Controllers/SearchController.php` (変更)

`init()` メソッドを追加:

- UUID生成 → `agent_conversations` に INSERT → `{conversation_id: uuid}` をJSONで返す
- 既存の `index()` / `query()` は変更しない

`routes/web.php` (変更)

```
Route::post('search/init', [SearchController::class, 'init'])->name('search.init');
```

3.4 Wayfinder 再生成

```
vendor/bin/sail artisan wayfinder:generate
```

→ `resources/js/routes/search/index.ts` に `init()` 関数が追加される

3.5 フロントエンド実装

`resources/js/components/search-dialog.tsx` (新規作成)

`search.tsx` のUIロジックをここに移植・拡張:

```
SearchDialog
├── isOpen state (boolean)
├── conversationId state (string | null) ← initエンドポイントから取得
├── messages state
├── isStreaming state
├── onOpenChange ハンドラ
├── open時: POST /search/init → conversationId をセット
├── Cmd+K useEffect (グローバル keydown)
├── handleSubmit (既存の fetch streaming ロジックをそのまま流用)
├── JSX
│   ├── Sheet (from @/components/ui/sheet)
│   │   ├── SheetTrigger: 検索アイコンボタン (app-headerから移動)
│   │   └── SheetContent (side="right", className="w-full sm:max-w-xl")
│   │       ├── SheetHeader: "スクラップを検索"
│   │       ├── メッセージリスト (flex-1 overflow-y-auto)
│   │       │   ├── 空状態: センター表示テキスト
│   │       │   └── messages.map (user: 右寄せバブル / assistant: MarkdownPreview)
│   │       ├── 入力フォーム (shrink-0 py-4)
│   │       │   ├── textarea + SendHorizontal Button
│   │       │   └── "Enter で送信・Shift+Enter で改行"
│   └── conversationId が null の間は init中スピナー表示
```

再利用コンポーネント:

- `MarkdownPreview` from `@/components/markdown-preview`
- `Spinner` from `@/components/ui/spinner`
- `Button` from `@/components/ui/button`
- `getXsrfToken()` ユーティリティ
- streaming fetch ロジック (`search.tsx` からほぼそのまま移植)
- `Wayfinder`: `init` from `@/routes/search`, `query` as `searchQuery` from `@/routes/search`

`resources/js/components/app-header.tsx` (変更)

- `import { SearchDialog } from '@components/search-dialog'`
- 現在の `<Link href={search()} prefetch><Button>...</Button></Link>` を `<SearchDialog />` に置き換え
- `search` import も削除

3.6 `/search` ページの扱い

`/search`

ルートはそのまま残す（既存ブックマーク・テスト互換性のため）。ページ自体の削除はしない。

3.7 テスト

`tests/Feature/SearchControllerTest.php` に追加:

- `test('search init creates conversation')` — POST `/search/init` が200 + `conversation_id` (UUID) を返すことを確認
- 認証なしで401を返すことを確認

4. Embeddingテキスト戦略の改善

4.1 背景

現在の `GenerateScrapEmbeddingJob` は `title + content` を丸ごと `embedding` に渡している。スクラップにはコードブロックが多く含まれることがあり、Amazon Titan（汎用テキストモデル）にとってコード構文はほぼノイズになる。

一方、`summary` フィールドには AI が生成した日本語の意味が凝縮されており、これを `embedding` テキストに使うことで検索精度が改善される。

4.2 Embeddingテキストの優先順位

1. `title + summary` (`summary` が存在する場合)
2. `title + コードブロックを除いた content` (`summary` が空の場合)

コードブロック除去は正規表現で ````` フェンスを `strip` する。

4.3 実装

`app/Jobs/GenerateScrapEmbeddingJob.php` (変更)

`handle()` メソッドの `embedding` テキスト組み立て部分を以下に変更：

```
$body = $scrap->summary
      ? $scrap->summary
      : $this->stripCodeBlocks($scrap->content ?? '');

$text = implode("¥n¥n", array_filter([
    $scrap->title,
    $body,
]));
```

`stripCodeBlocks()` プライベートメソッドを追加：

```
private function stripCodeBlocks(string $content): string
{
    return trim(preg_replace('/```[¥s¥S]*?```/m', '', $content));
}
```

4.4 テスト

`tests/Feature/GenerateScrapEmbeddingJobTest.php` に追加：

- `summary` あり → `summary` がテキストに使われること (`content` のコードが除外されること)を確認
- `summary` なし・`content` にコードブロックあり → コードブロックが `strip` された `content` が使われること
- `summary` なし・`content` にコードなし → `content` がそのまま使われること

4.5 既存データの再生成

```
# embeddingをリセット
UPDATE scraps SET embedding = NULL;
```

```
# 再生成実行
vendor/bin/sail artisan scraps:embed
```

5. ダッシュボードのスクラップ一覧統合とアーカイブフィルタ化

5.1 背景

アーカイブ専用ページを廃止し、スクラップ一覧を `dashboard` に統合する。初期表示は非アーカイブのトップレベルスクラップのみとし、`status=archived` フィルタでアーカイブ済みトップレベルスクラップを表示できるようにする。

5.2 実装方針

- `DashboardController` に `status` クエリフィルタを追加する
- 一覧・詳細・タグ絞り込み・子スクラップ取得を `active` / `archived` の表示状態に合わせて切り替える
- `resources/js/pages/dashboard.tsx` に `Active / Archived` 切替UIを追加する
- アーカイブ詳細では `復元` と `完全削除` を表示し、非アーカイブ詳細では従来どおり `アーカイブへ送る` を表示する
- サイドバーとヘッダーから `Archives` 導線を削除する
- `ArchivesController` は専用表示をやめて `dashboard?status=archived` へのリダイレクトに置き換える
- `resources/js/pages/archives.tsx` は不要になるため削除する
- `ScrapController::destroy()` の削除後遷移先を `dashboard?status=archived` に変更する

5.3 リダイレクト

既存の `/archives` は互換導線として `/dashboard?status=archived` へリダイレクトする。

5.4 テスト

- `DashboardTest`
 - 通常表示で非アーカイブのトップレベルのみ出ること
 - `status=archived` でアーカイブ済みトップレベルのみ出ること
 - `status=archived` の詳細表示が開けること
- `ArchivesTest`
 - `/archives` が `dashboard?status=archived` へリダイレクトすること

- StoreScrapTest
 - 完全削除後の遷移先が `dashboard?status=archived` になること

5.5 前提条件

- 一覧対象はトップレベルスクラップのみとし、子スクラップは詳細画面で扱う
 - `status` フィルタは `active` と `archived` の2値のみ扱う
-

6. Codex向けプラン保存フロー移植

6.1 目的

既存のClaude向け `plan/result`

保存フローをCodexでも使えるようにし、このLaravelアプリ内で承認済みプラン保存と実行結果の子scrap

6.2 実装手順

1. `plans:save` と `plans:result`
を汎用化し、呼び出し元ワークフローを識別できるようにする
2. `.codex/skills/` 配下にCodex向けskill定義とシェルスクリプトを追加する
3. シェルスクリプトを `Sail` と `php artisan` の両方で動くようにする
4. 新しいメタデータ挙動に対するPestテストを追加する
5. プランを保存し、実装結果を子scrapとして紐付ける

6.3 リスク

- `Sail`前提のファイルパス仮定があると、非コンテナ環境で破綻する
 - `skill`の配置場所はローカルCodexランタイムが読める構成と一致している必要がある
-

7. 変更ファイル一覧（全体）

ファイル	変更種別	関連セクション
<code>app/Ai/Agents/SearchScrapAgent.php</code>	新規作成	2
<code>app/Http/Controllers/SearchController.php</code>	新規作成 → <code>init()</code> 追加	2, 3
<code>app/Http/Requests/SearchQueryRequest.php</code>	新規作成	2
<code>routes/web.php</code>	ルート追加	2, 3

ファイル	変更種別	関連セクション
<code>resources/js/pages/search.tsx</code>	新規作成	2
<code>resources/js/components/search-dialog.tsx</code>	新規作成	3
<code>resources/js/components/app-header.tsx</code>	変更	2, 3
<code>resources/js/routes/search/index.ts</code>	wayfinder自動生成	3
<code>tests/Feature/SearchControllerTest.php</code>	テスト追加	3
<code>app/Jobs/GenerateScrapEmbeddingJob.php</code>	変更	4
<code>tests/Feature/GenerateScrapEmbeddingJobTest.php</code>	テスト追加	4
<code>app/Http/Controllers/DashboardController.php</code>	変更	5
<code>resources/js/pages/dashboard.tsx</code>	変更	5
<code>app/Http/Controllers/ArchivesController.php</code>	リダイレクトに変更	5
<code>resources/js/pages/archives.tsx</code>	削除	5
<code>.codex/skills/</code>	新規作成	6

8. 検証手順

共通コマンド

```
# テスト実行
vendor/bin/sail artisan test --compact --filter=SearchController
vendor/bin/sail artisan test --compact --filter=GenerateScrapEmbeddingJob
vendor/bin/sail artisan test --compact --filter=Dashboard
vendor/bin/sail artisan test --compact --filter=Archives

# コードフォーマット
vendor/bin/sail bin pint --dirty --format agent

# Wayfinder再生成
vendor/bin/sail artisan wayfinder:generate
```

ブラウザ確認項目

1. どのページでも検索アイコンクリック → Sheet が右からスライドイン

2. `Cmd+K` / `Ctrl+K` でも同様に開く
3. クエリ入力 → ストリーミングレスポンスが表示される
4. Sheet を閉じて別ページに移動し、再度開く → 新しい会話 (`conversationId` リセット)
5. `/search` ページへの直接アクセスが引き続き動作する
6. マルチターンの文脈保持が機能する
7. ダッシュボードで Active / Archived の切り替えが正常に動作する
8. `/archives` が `dashboard?status=archived` にリダイレクトされる
9. Related scraps が summary ベースの embedding で正しく表示される